

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010

OPERATING SYSTEM

WEEK 11 LECTURE 1

**INTER PROCESS
COMMUNICATION**

WISH YOU ALL THE BEST

System V IPC

- ★ **IPC is an abbreviation for Inter process Communication and commonly refers to a set of mechanisms that allow a User Mode process to do the following:**
 - ❖ **Synchronize itself with other processes by means of semaphores**
 - ❖ **Send messages to other processes or receive messages from them**
 - ❖ **Share a memory area with other processes**
- ★ **System V IPC first appeared in a development Unix variant called "Columbus Unix " and later was adopted by AT&T's System III .**
- ★ **It is now found in most Unix systems, including Linux.**

System V IPC

- ★ IPC data structures are created dynamically when a process requests an IPC resource (a semaphore, a message queue, or a shared memory region).
- ★ An IPC resource is persistent: unless explicitly removed by a process, it is kept in memory and remains available until the system is shut down.
- ★ An IPC resource may be used by every process, including those that do not share the ancestor that created the resource.
- ★ Because a process may require several IPC resources of the same type, each new resource is identified by a 32-bit IPC key, which is similar to the file pathname in the system's directory tree.
- ★ Each IPC resource also has a 32-bit IPC identifier, which is somewhat similar to the file descriptor associated with an open file.
- ★ IPC identifiers are assigned to IPC resources by the kernel and are unique within the system, while IPC keys can be freely chosen by programmers.
- ★ When two or more processes wish to communicate through an IPC resource, they all refer to the IPC identifier of the resource.

□ Using an IPC Resource

- ★ IPC resources are created by invoking the **semget()**, **msgget()**, or **shmget()** functions, depending on whether the new resource is a semaphore, a message queue, or a shared memory region.
- ★ The main objective of each of these three functions is to derive from the IPC key (passed as the first parameter) the corresponding IPC identifier, which is then used by the process for accessing the resource.
- ★ If there is no IPC resource already associated with the IPC key, a new resource is created.
- ★ If everything goes right, the function returns a positive IPC identifier; otherwise, it returns one of the error codes.

System V IPC

- ★ IPC identifier assigned to a resource is almost always larger than the identifier assigned to the previously allocated resource of the same type.
- ★ Each IPC identifier is computed by **combining a slot usage sequence number relative to the resource type**, an arbitrary slot index for the allocated resource, **and** an arbitrary value chosen in the kernel that is greater than the maximum number of allocatable resources.
- ★ If we choose s to represent the slot usage sequence number, M to represent the upper bound on the number of allocatable resources, and i to represent the slot index, where $0 \leq i < M$, each IPC resource's ID is computed as follows:

$$\text{IPC identifier} = s \times M + i$$

- ★ In Linux, the value of M is set to 32,768 (IPCMNI macro). The slot usage sequence number s is initialized to 0 and is increased by 1 at every resource allocation.
- ★ When s reaches a predefined threshold, which depends on the type of IPC resource, it restarts from 0.

System V IPC

- ★ Every type of IPC resource (semaphores, message queues, and shared memory areas) owns an **ipc_ids data structure**, which includes the fields shown in Table

Table - fields of the ipc_ids data structure

<u>Type</u>	<u>Field</u>	<u>Description</u>
Int	in_use	Number of allocated IPC resources
Int	max_id	Maximum slot index in use
unsigned short	seq	Slot usage sequence number for the next allocation
unsigned short	seq_max	Maximum slot usage sequence number
struct semaphore	sem	Semaphore protecting the ipc_ids data structure
struct ipc_id_ary	nullentry	Fake data structure pointed to by the entries field if this IPC resource cannot be initialized (normally not used)
struct ipc_id_ary * enTRies resource		Pointer to the ipc_id_ary data structure for this resource

ipc() System Call

- ★ All IPC functions must be implemented through suitable Linux system calls.
- ★ Actually, in the 80 x 86 architecture, there is just one IPC system call named **ipc()**.
- ★ When a process invokes an IPC function, let's say **msgget()**, it really invokes **a wrapper function in the C library**.
- ★ This in turn invokes the **ipc()** system call by passing to it all the **parameters of msgget()** plus a proper subcommand code in this case, **MSGGET**.
- ★ The **sys_ipc()** service routine examines the subcommand code and invokes the kernel function that implements the requested service.
- ★ As a matter of fact, Linux provides one system call for each IPC function on Hewlett-Packard's Alpha architecture and on Intel's IA-64.

IPC Semaphores

- ★ **IPC semaphores are counters used to provide controlled access to shared data structures for multiple processes.**
- ★ **The semaphore value is positive if the protected resource is available, and 0 if the protected resource is currently not available.**
- ★ **A process that wants to access the resource tries to decrease the semaphore value; the kernel, however, blocks the process until the operation on the semaphore yields a positive value.**
- ★ **When a process relinquishes a protected resource, it increases its semaphore value; in doing so, any other process waiting for the semaphore is woken up.**

IPC Semaphores

- ★ Actually, IPC semaphores are more complicated to handle than kernel semaphores for two main reasons:
- ★ Each IPC semaphore is a set of one or more semaphore values, not just a single value like a kernel semaphore.
- ★ This means that the same IPC resource can protect several independent shared data structures.
- ★ The number of semaphore values in each IPC semaphore must be specified as a parameter of the `semget( )` function when the resource is being allocated.
- ★ The counters inside an IPC semaphore are referred to as **primitive semaphores**.
- ★ There are bounds both on the number of IPC semaphore resources (by default, 128) and on the number of primitive semaphores inside a single IPC semaphore resource (by default, 250); however, the system administrator can easily modify these bounds by writing into the `/proc /sys/kernel/sem` file.

IPC Semaphores

- ★ **System V IPC semaphores provide a fail-safe mechanism for situations in which a process dies without being able to undo the operations that it previously issued on a semaphore.**
- ★ **When a process chooses to use this mechanism, the resulting operations are called undoable semaphore operations.**
- ★ **When the process dies, all of its IPC semaphores can revert to the values they would have had if the process had never started its operations.**
- ★ **This can help prevent other processes that use the same semaphores from remaining blocked indefinitely as a consequence of the terminating process failing to manually undo its semaphore operations.**

IPC Semaphores

- * Typical steps performed by a process wishing to access one or more resources protected by an IPC semaphore:
 - * Invokes the **semget()** wrapper function to get the IPC semaphore identifier, specifying as the parameter the IPC key of the IPC semaphore that protects the shared resources.
 - * If the process wants to **create a new IPC semaphore**, it also **specifies the IPC_CREATE or IPC_PRIVATE flag and the number of primitive semaphores required.**
 - * Invokes the **semop()** wrapper function to test and decrease all primitive semaphore values involved.
 - * If all the tests succeed, the decrements are performed, the function terminates, and the process is allowed to access the protected resources.
 - * If some semaphores are in use, the process is usually suspended until some other process releases the resources.

IPC Semaphores

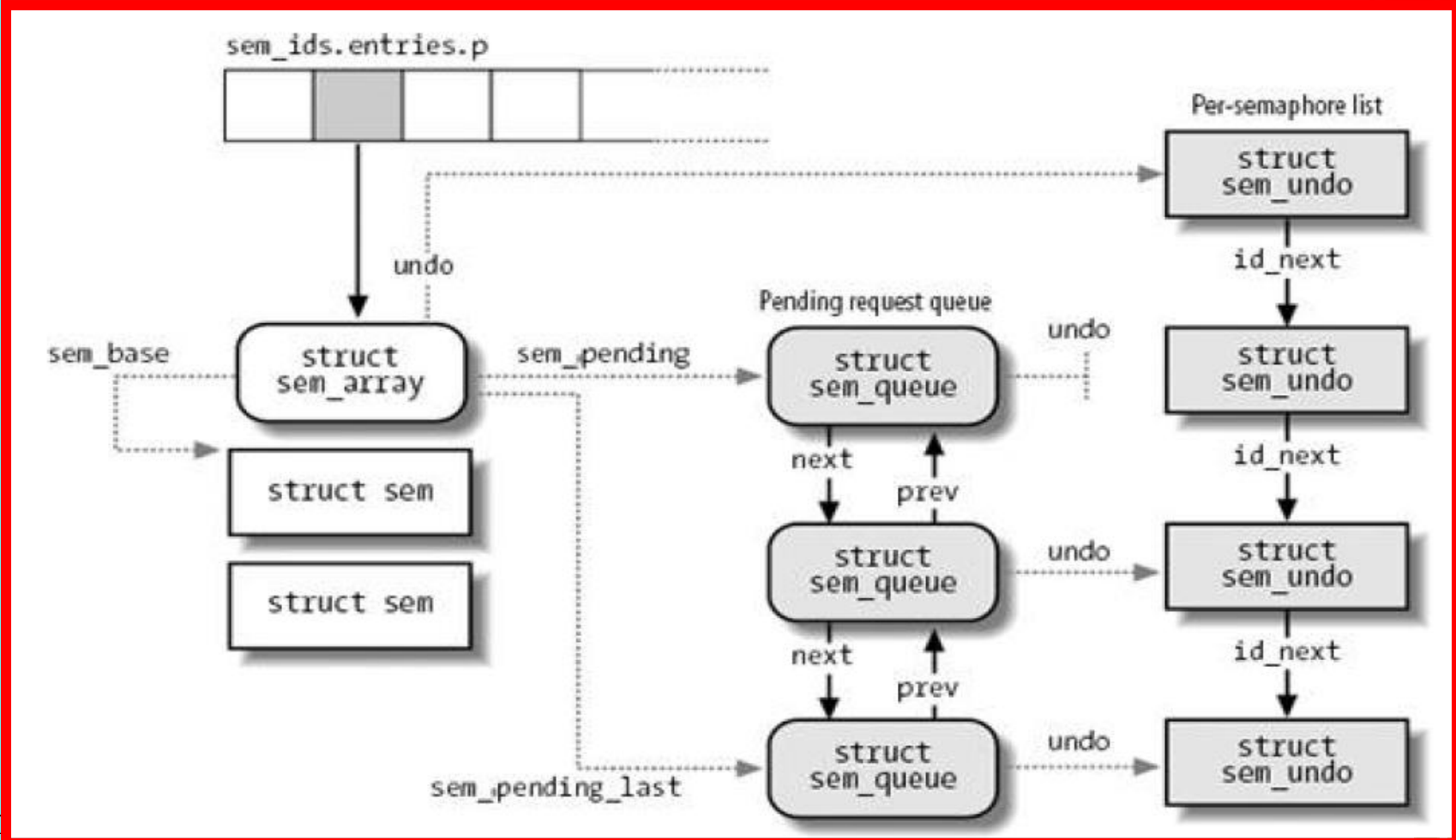
- ★ The function receives as its parameters the IPC semaphore identifier, an array of integers specifying the operations to be atomically performed on the primitive semaphores, and the number of such operations.
- ★ Optionally, the process may specify the SEM_UNDO flag, which instructs the kernel to reverse the operations, should the process exit without releasing the primitive semaphores.
- ★ When relinquishing the protected resources, it invokes the semop() function again to atomically increase all primitive semaphores involved.
- ★ Optionally, it invokes the semctl() wrapper function, specifying the IPC_RMID command to remove the IPC semaphore from the system.

Phoneme array of source.

IPC semaphore data structures

- * The `sem_ids` variable stores the `ipc_ids` data structure of the IPC semaphore resource type; the corresponding `ipc_id_ary` data structure contains an array of pointers to `sem_array` data structures, one item for every IPC semaphore resource.

- ## IPC semaphore data structures
- * The `sem_ids` variable stores the `ipc_ids` data structure of the IPC semaphore resource type; the corresponding `ipc_id_ary` data structure contains an array of pointers to `sem_array` data structures, one item for every IPC semaphore resource.



IPC Semaphores

- * Formally, the array stores pointers to **kern_ipc_perm** data structures, but each structure is simply the first field of the **sem_array** data structure.
- * All fields of the **sem_array** data structure are shown in Table

Table - fields in the sem_array data structure

Type	Field	Description
struct kern_ipc_perm	sem_perm	kern_ipc_perm data structure
long	sem_otime	Timestamp of last semop()
Long	sem_ctime	Timestamp of last change
struct sem *	sem_base	Pointer to first sem structure
struct sem_queue *	sem_pending	Pending operations
struct sem_queue **	sem_pending_last	Last pending operation
struct sem_undo *	undo	Undo requests
unsigned long	sem_nsems	Number of semaphores in array

The **sem_base** field points to an array of **sem data structures**, one for every IPC primitive semaphore. The latter data structure includes only two fields:

Semval The value of the semaphore's counter.

Sempid The PID of the last process that accessed the semaphore. This value can be queried by a process through the **semctl()** wrapper function

IPC Semaphores

□ Undoable semaphore operations

- * If a process aborts suddenly, it cannot undo the operations that it started (for instance, release the semaphores it reserved); so by declaring them undoable, the process lets the kernel return the semaphores to a consistent state and allow other processes to proceed.
- * Processes can request undoable operations by specifying the SEM_UNDO flag in the semop() function.
- * Information to help the kernel reverse the undoable operations performed by a given process on a given IPC semaphore resource is stored in a sem_undo data structure.
- * It essentially contains the IPC identifier of the semaphore and an array of integers representing the changes to the primitive semaphore's values caused by all undoable operations performed by the process.
- * For each process, the kernel keeps track of all semaphore resources handled with undoable operations so that it can roll them back if the process unexpectedly exits.
- * Furthermore, for each semaphore, the kernel has to keep track of all its sem_undo structures so it can quickly access them whenever a process uses semctl() to force an explicit value into a primitive semaphore's counter or to destroy an IPC semaphore resource.

□ Undoable semaphore operations

- * The kernel is able to handle undo tasks efficiently, by maintaining two lists, which are denoted as the **per-process** and the **per-semaphore lists**.
- * The first list keeps track of all semaphores operated upon by a given process with undoable operations.
- * The second list keeps track of all processes that are acting on a given semaphore with undoable operations.
- * The **per-process list** includes all `sem_undo` data structures corresponding to IPC semaphores on which the process has performed undoable operations.
- * The `sysvsem.undo_list` field of the process descriptor points to a data structure, of type `sem_undo_list`, which in turn contains a pointer to the first element of the list; the `proc_next` field of each `sem_undo` data structure points to the next element in the list.
- * The **per-semaphore list** includes all `sem_undo` data structures corresponding to the processes that performed undoable operations on the semaphore.
- * The `undo` field of the `sem_array` data structure points to the first element of the list, while the `id_next` field of each `sem_undo` data structure points to the next element in the list.

IPC Semaphores

□ Undoable semaphore operations

- ★ The **per-process list** is used when a process terminates.
- ★ The **exit_sem()** function, which is invoked by **do_exit()**, walks through the list and reverses the effect of any unbalanced operation for every IPC semaphore touched by the process.
- ★ By contrast, the per-semaphore list is mainly used when a process invokes the **semctl()** function to force an explicit value into a primitive semaphore.
- ★ The kernel sets the corresponding element to 0 in the arrays of all **sem_undo** data structures referring to that IPC semaphore resource, because it would no longer make any sense to reverse the effect of previous undoable operations performed on that primitive semaphore.
- ★ Moreover, the per-semaphore list is also used when an IPC semaphore is destroyed; all related **sem_undo** data structures are invalidated by setting the **semid** field to -1.

IPC Semaphores

❑ Queue of pending requests

- ★ The kernel associates a queue of pending requests with each IPC semaphore to identify processes that are waiting on one (or more) of the semaphores in the array.
- ★ The queue is a doubly linked list of `sem_queue` data structures whose fields are shown in Table.
- ★ The first and last pending requests in the queue are referenced, respectively, by the `sem_pending` and `sem_pending_last` fields of the `sem_array` structure.
- ★ This last field allows the list to be handled as easily as a FIFO; new pending requests are added to the end of the list so they will be serviced later.
- ★ The most important fields of a pending request are `nsops` (which stores the number of primitive semaphores involved in the pending operation) and `sops` (which points to an array of integer values describing each semaphore operation).
- ★ The `sleeper` field stores the descriptor address of the sleeping process that requested the operation.

IPC Semaphores

❑ Queue of pending requests

Table - Fields in the sem_queue data structure

<u>Type</u>	<u>Field</u>	<u>Description</u>
struct sem_queue *	next	Pointer to next queue element
struct sem_queue **	prev	Pointer to previous queue element
struct task_struct *	sleeper	Pointer to the sleeping process that requested the semaphore operation
struct sem_undo *	undo	Pointer to sem_undo structure
Int	pid	Process identifier
Int	status	Completion status of operation
struct sem_array *	sma	Pointer to IPC semaphore descriptor
Int	id	Slot index of the IPC semaphore resource
struct sembuf *	sops	Pointer to array of pending operations
Int	nsops	Number of pending operations
Int	alter	Flag denoting whether the operation modifies the semaphore array

IPC Messages

- ❑ **IPC Messages**
- ❑ **Processes can communicate with one another by means of IPC messages**
- ❑ **Each message generated by a process is sent to an IPC message queue, where it stays until another process reads it.**
- ❑ **A message is composed of a fixed-size header and a variable-length text; it can be labeled with an integer value (the message type), which allows a process to selectively retrieve messages from its message queue.**
- ❑ **Once a process has read a message from an IPC message queue, the kernel destroys the message; therefore, only one process can receive a given message.**
- ❑ **The message queue is implemented by means of a linked list.**
- ❑ **Because messages can be retrieved in an order different from "first in, first out," the name "message queue" is not appropriate.**
- ❑ **However, new messages are always put at the end of the linked list.**

IPC Messages

- ❑ **IPC Messages**
- ❑ **To send a message, a process invokes the `msgsnd( )` function, passing the following as parameters:**
 - The IPC identifier of the destination message queue
 - The size of the message text
 - The address of a User Mode buffer that contains the message type immediately followed by the message text
- ❑ **To retrieve a message, a process invokes the `msgrcv( )` function, passing to it:**
 - The IPC identifier of the IPC message queue resource
 - The pointer to a User Mode buffer to which the message type and message text should be copied
 - The size of this buffer
 - A value `t` that specifies what message should be retrieved
 - If the value `t` is 0, the first message in the queue is returned.
 - If `t` is positive, the first message in the queue with its type equal to `t` is returned.
 - Finally, if `t` is negative, the function returns the first message whose message type is the lowest value less than or equal to the absolute value of `t`.

IPC Messages

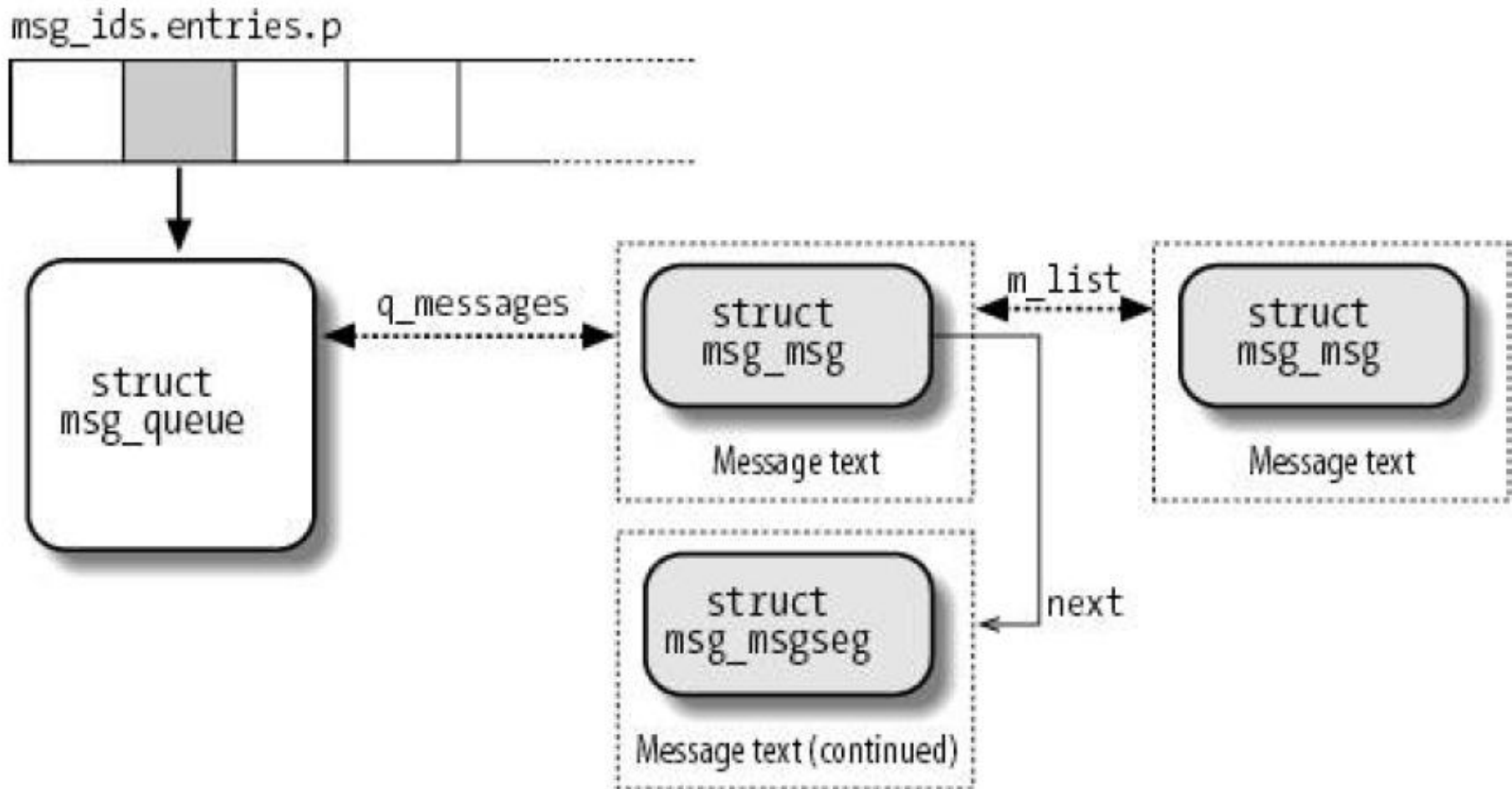
- ❑ **IPC Messages**
- ❑ **To send a message, a process invokes the `msgsnd( )` function, passing the following as parameters:**
 - The IPC identifier of the destination message queue
 - The size of the message text
 - The address of a User Mode buffer that contains the message type immediately followed by the message text
- ❑ **To retrieve a message, a process invokes the `msgrcv( )` function, passing to it:**
 - The IPC identifier of the IPC message queue resource
 - The pointer to a User Mode buffer to which the message type and message text should be copied
 - The size of this buffer
 - A value `t` that specifies what message should be retrieved
 - If the value `t` is 0, the first message in the queue is returned.
 - If `t` is positive, the first message in the queue with its type equal to `t` is returned.
 - Finally, if `t` is negative, the function returns the first message whose message type is the lowest value less than or equal to the absolute value of `t`.

IPC Messages

- ❑ To avoid resource exhaustion, there are some limits on the number of IPC message queue resources allowed (by default, 16), on the size of each message (by default, 8,192 bytes), and on the maximum total size of the messages in a queue (by default, 16,384 bytes).
- ❑ As usual, however, the system administrator can tune these values by writing into the `/proc/sys/kernel/msgmni`, `/proc/sys/kernel/msgmnb`, and `/proc/sys/kernel/msgmax` files, respectively.

IPC Messages

- ❑ The data structures associated with IPC message queues are shown in Figure
- ❑ The `msg_ids` variable stores the `ipc_ids` data structure of the IPC message queue resource type; the corresponding `ipc_id_ary` data structure contains an array of pointers to `shmid_kernel` data structures one item for every IPC message queue resource.



IPC Messages



- Formally, the array stores pointers to `kern_ipc_perm` data structures, but each such structure is simply the first field of the `msg_queue` data structure. All fields of the `msg_queue` data structure are shown in Table.
- Table The `msg_queue` data structure**

Type	Field	Description
<code>struct kern_ipc_perm</code>	<code>q_perm</code>	<code>kern_ipc_perm</code> data structure
Long	<code>q_stime</code>	Time of last <code>msgsnd()</code>
Long	<code>q_rtime</code>	Time of last <code>msgrcv()</code>
Long	<code>q_ctime</code>	Last change time
unsigned long	<code>q_qcbytes</code>	Number of bytes in queue
unsigned long	<code>q_qnum</code>	Number of messages in queue
unsigned long	<code>q_qbytes</code>	Maximum number of bytes in queue
Int	<code>q_lspid</code>	PID of last <code>msgsnd()</code>
Int	<code>q_lrpid</code>	PID of last <code>msgrcv()</code>
<code>struct list_head</code>	<code>q_messages</code>	List of messages in queue
<code>struct list_head</code>	<code>q_receivers</code>	List of processes receiving messages
<code>struct list_head</code>	<code>q_senders</code>	List of processes sending messages

The most important field is **`q_messages`**, which represents the head (i.e., the first dummy element) of a doubly linked circular list containing all messages currently in the queue.

IPC Messages

- ❑ Each message is broken into one or more pages, which are dynamically allocated.
- ❑ The beginning of the first page stores the message header, which is a data structure of type `msg_msg`; its fields are listed in Table.
- ❑ The `m_list` field stores the pointers to the previous and next messages in the queue.
- ❑ The message text starts right after the `msg_msg` descriptor; if the message is longer than 4,072 bytes (the page size minus the size of the `msg_msg` descriptor), it continues on another page, whose address is stored in the next field of the `msg_msg` descriptor.
- ❑ The second page frame starts with a descriptor of type `msg_msgseg`, which simply includes a next pointer storing the address of an optional third page, and so on.

IPC Messages

□ Table - The msg_msg data structure

<u>Type</u>	<u>Field</u>	<u>Description</u>
struct list_head	m_list	Pointers for message list
Long	m_type	Message type
Int	m_ts	Message text size
struct msg_msgseg *	next	Next portion of the message
void *	security	Pointer to a security data structure (used by SELinux)

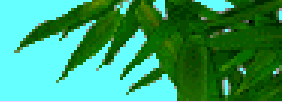
IPC Messages

- ❑ **When the message queue is full** (either the maximum number of messages or the maximum total size has been reached), **processes that try to enqueue new messages may be blocked.**
- ❑ The `q_senders` field of the `msg_queue` data structure is the head of a list that includes the pointers to the descriptors of all blocked sending processes.
- ❑ **Even receiving processes may be blocked when the message queue is empty** (or the process specified a type of message not present in the queue).
- ❑ The `q_receivers` field of the `msg_queue` data structure is the head of a list of `msg_receiver` data structures, one for every blocked receiving process.
- ❑ Each of these structures essentially includes a pointer to the process descriptor, a pointer to the `msg_msg` structure of the message, and the type of the requested message.

IPC Shared Memory

- ❖ The most useful IPC mechanism is shared memory , which allows two or more processes to access some common data structures by placing them in an IPC shared memory region.
- ❖ Each process that wants to access the data structures included in an IPC shared memory region must add to its address space a new memory region, which maps the page frames associated with the IPC shared memory region.
- ❖ Such page frames can then be easily handled by the kernel through demand paging

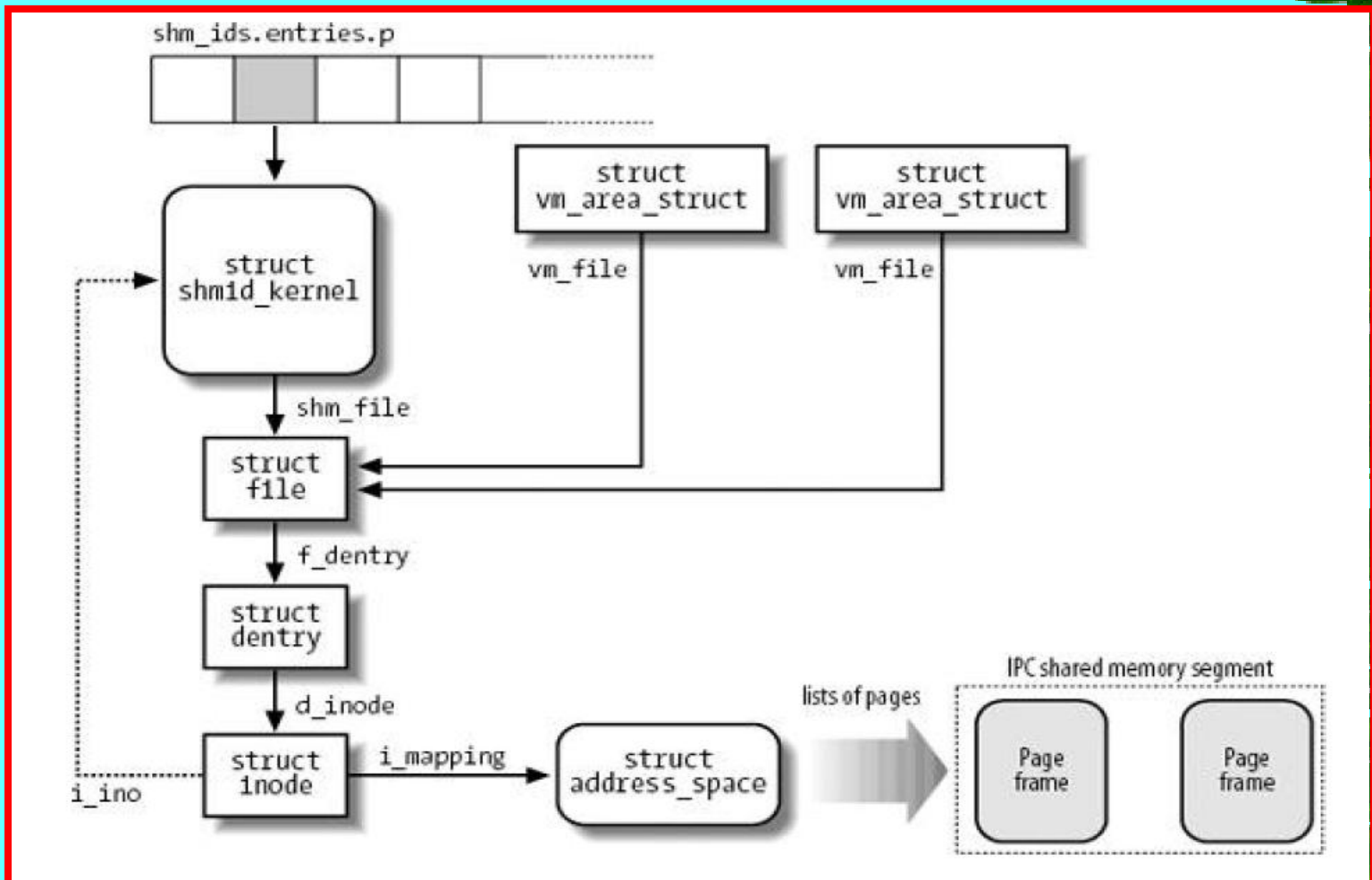
IPC Shared Memory



- As with semaphores and message queues, the **shmget()** function is invoked to get the IPC identifier of a shared memory region, optionally creating it if it does not already exist.
- The **shmat()** function is invoked to "attach" an IPC shared memory region to a process. It receives as its parameter the identifier of the IPC shared memory resource and tries to add a shared memory region to the address space of the calling process
- The calling process can require a specific starting linear address for the memory region, but the address is usually unimportant, and each process accessing the shared memory region can use a different address in its own address space.
- The process's Page Tables are left unchanged by **shmat()**.
- The **shmdt()** function is invoked to "detach" an IPC shared memory region specified by its IPC identifier that is, to remove the corresponding memory region from the process's address space.
- Recall that an IPC shared memory resource is persistent: even if no process is using it, the corresponding pages cannot be discarded, although they can be swapped out.

IPC Shared Memory

- ❑ The data structures associated with IPC shared memory regions are shown in Figure.
- ❑ The `shm_ids` variable stores the `ipc_ids` data structure of the IPC shared memory resource type; the corresponding `ipc_id_ary` data structure contains an array of pointers to `shmid_kernel` data structures, one item for every IPC shared memory resource. Formally, the array stores pointers to `kern_ipc_perm` data structures, but each such structure is simply the first field of the `msg_queue` data structure.



IPC Shared Memory

❑ Table - The fields in the shmid_kernel data structure

<u>Type</u>	<u>Field</u>	<u>Description</u>
struct kern_ipc_perm	shm_perm	kern_ipc_perm data structure
struct file *	shm_file	Special file of the segment
Int	id	Slot index of the segment
unsigned long	shm_nattch	Number of current attaches
unsigned long	shm_segsz	Segment size in bytes
Long	shm_atim	Last access time
Long	shm_dtimLast	detach time
Long	shm_ctim	Last change time
Int	shm_cprid	PID of creator
Int	shm_lprid	PID of last accessing process
struct user_struct *	mlock_user	Pointer to the user_struct descriptor of the user that locked in RAM the shared memory resource

IPC Shared Memory

- ❑ The most important field is **shm_file**, which stores the address of a file object. This reflects the tight integration of IPC shared memory with the VFS layer in Linux
- ❑ In particular, each IPC shared memory region is associated with a file belonging to the **shm special filesystem**
- ❑ Because the shm filesystem has no mount point in the system directory tree, no user can open and access its files by means of regular VFS system calls.
- ❑ However, when a process "attaches" a segment, the kernel invokes **do_mmap()** and creates a new shared memory mapping of the file in the address space of the process.
- ❑ Therefore, files that belong to the shm special filesystem have just one file object method, **mmap**, which is implemented by the **shm_mmap()** function.

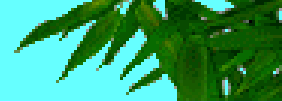
IPC Shared Memory

- ❑ A memory region that corresponds to an **IPC shared memory region** is described by a **vm_area_struct** object, its **vm_file** field points back to the file object of the file in the special filesystem, which in turn references a dentry object and an inode object.
- ❑ The inode number, stored in the **i_ino** field of the inode, is actually the slot index of the IPC shared memory region, so the inode object indirectly references the **shmid_kernel** descriptor.
- ❑ As usual for every shared memory mapping, page frames are included in the page cache through an **address_space** object, which is embedded in the inode and referenced by the **i_mapping** field of the inode; in case of page frames belonging to an IPC shared memory region, the methods of the **address_space** object are stored in the **shmem_aops** global variable.

IPC Shared Memory

- ❑ **Swapping out pages of IPC shared memory regions**
- ❑ The kernel has to be careful when swapping out pages included in shared memory regions, and the role of the swap cache is crucial
- ❑ Pages of an IPC shared memory region are swappable and not syncable because they map a special inode that has no image on disk.
- ❑ Thus, in order to reclaim a page of an IPC shared memory region, the kernel must write it into a swap area.
- ❑ Because an IPC shared memory region is persistent that is, its pages must be preserved even when the segment is not attached to any process the kernel cannot simply discard these pages even when they are no longer used by any process.

IPC Shared Memory



❑ Demand paging for IPC shared memory regions

- The pages added to a process by **shmat()** are dummy pages; the function adds a new memory region into a process's address space, but it doesn't modify the process's Page Tables.
- Moreover, pages of an IPC shared memory region can be swapped out. Therefore, these pages are handled through the demand paging mechanism.
- A Page Fault occurs when a process tries to access a location of an IPC shared memory region whose underlying page frame has not been assigned.
- The corresponding exception handler determines that the faulty address is inside the process address space and that the corresponding Page Table entry is null; therefore, it invokes the **do_no_page()** function
- In turn, this function checks whether the nopage method for the memory region is defined.
- That method is invoked, and the Page Table entry is set to the address returned from it
- Memory regions used for IPC shared memory always define the **nopage method**.



IPC Shared Memory



❑ Demand paging for IPC shared memory regions

nopage method is implemented by the **shmem_nopage()** function, which performs the following operations:

- Walks the chain of pointers in the VFS objects and derives the address of the inode object of the IPC shared memory resource
- Computes the logical page number inside the segment from the **vm_start** field of the memory region descriptor and the requested address.
- Checks whether the page is already included in the page cache; if so, terminates by returning the address of its descriptor.
- Checks whether the page is included in the swap cache and is up-to-date; if so, terminates by returning the address of its descriptor.
- Checks whether the **shmem_inode_info** that embodies the inode object stores a swapped-out page identifier for the logical page number. If so, it performs a swap-in operation by invoking **read_swap_cache_async()**, waits until the data transfer completes, and terminates by returning the address of the page descriptor.
- Otherwise, the page is not stored in a swap area; therefore, the function allocates a new page from the buddy system, inserts it into the page cache, and returns its address.
- The **do_no_page() function** sets the entry that corresponds to the faulty address in the process's Page Table so that it points to the page frame returned by the method.

POSIX Message Queues

❑ POSIX Message Queues

- The POSIX standard (IEEE Std 1003.1-2001) defines an IPC mechanism based on message queues, which is usually known as POSIX message queues .
- They are much like the System V IPC's message queues
- However, POSIX message queues sport a number of advantages over the older queues:
 - A much simpler file-based interface to the applications
 - Native support for message priorities (the priority ultimately determines the position of the message in the queue)
 - Native support for asynchronous notification of message arrivals, either by means of signals or thread creation
 - Timeouts for blocking send and receive operations

POSIX Message Queues

❑ Table -Library functions for POSIX message queues

<u>Function names</u>	<u>Description</u>
mq_open()	Open (optionally creating) a POSIX message queue
mq_close()	Close a POSIX message queue (without destroying it)
mq_unlink()	Destroy a POSIX message queue mq_send() ,
mq_send()	Send a message to a POSIX message queue; the latter function defines a time limit for the operation
mq_timedsend()	
mq_receive() ,	
mq_timedreceive()	Fetch a message from a POSIX message queue; the latter function defines a time limit for the operation
mq_notify()	Establish an asynchronous notification mechanism for the arrival of messages in an empty POSIX message queue
mq_getattr() ,	Respectively get and set attributes of a POSIX message queue (essentially, whether the send and receive operations should be blocking or nonblocking)
mq_setattr()	

POSIX Message Queues

- ❑ Application typically makes use of THE LIBRARY functions as follows :
- As a first step, the application invokes the **mq_open()** library function to open a POSIX message queue.
- The first argument of the function is a string specifying the name of the queue; it is similar to a filename, and indeed it must start with a slash (/).
- The library function accepts a subset of the flags of the **open()** system call: **O_RDONLY**, **O_WRONLY**, **O_RDWR**, **O_CREAT**, **O_EXCL**, and **O_NONBLOCK** (for nonblocking send and receive operations).
- Notice that the application may create a new POSIX message queue by specifying the **O_CREAT** flag.
- The **mq_open()** function returns a descriptor for the queue much like the file descriptor returned by the **open()** system call.

POSIX Message Queues

- ❑ Application typically makes use of THE LIBRARY functions as follows :
- Once a POSIX message queue has been opened, the application may send and receive messages by using the library functions `mq_send( )` and `mq_receive( )`, passing to them the queue descriptor returned by `mq_open( )`.
- The application may also make use of `mq_timedsend( )` and `mq_timedreceive( )` to specify the maximum time that the application will spend waiting for the send or receive operation to complete.

POSIX Message Queues

- ❑ Application typically makes use of THE LIBRARY functions as follows :
- Rather than blocking in `mq_receive()` or continuously polling the message queue if the `O_NONBLOCK` flag was specified the application might also establish an asynchronous notification mechanism by executing the `mq_notify( )` library function.
- Essentially, the application may require that when a message is inserted in an empty queue, either a signal is sent to a selected process, or a new thread is created.

POSIX Message Queues

- ❑ Application typically makes use of THE LIBRARY functions as follows :

Finally, when the application has finished using the message queue, it invokes the **mq_close()** library function; passing to it the queue descriptor.

Notice that this function does not destroy the queue, exactly as the **close() system call** does not remove a file.

To destroy a queue, the application makes use of the **mq_unlink()** function.

POSIX Message Queues

- ❑ Application typically makes use of THE LIBRARY functions as follows :

Finally, when the application has finished using the message queue, it invokes the **mq_close()** library function; passing to it the queue descriptor.

Notice that this function does not destroy the queue, exactly as the **close() system call** does not remove a file.

To destroy a queue, the application makes use of the **mq_unlink()** function.

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 19 of Text Book

Text Book

➤ Understanding the Linux Kernel

- Daniel P/ Bovet and Marco Cesati
Edition, O' Reilly Media 2005, ISBN 978-0596005658**

-

SOC 3010

OPERATING SYSTEM

END OF

WEEK 11 LECTURE 1

INTER PROCESS

COMMUNICATION

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

